

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
คณะเทคโนโลยีสารสนเทศ



รายงานโครงงานวิชา Cloud Technology (ภาคเรียนที่ 1/2569)

DISNEY LORCANA PLAYLAB CLOUD

ระบบจำลองห้องเล่นและวิเคราะห์ได้การ์ดแบบเรียลไทม์บนคลาวด์ Serverless

(สถานะความก้าวหน้า: เสร็จสิ้น Sprint 1–3 พร้อมส่งมอบ Stage 2)

กลุ่มที่: G21

สาขาวิชา: เทคโนโลยีสารสนเทศ (Multimedia & Game Development / Software Engineering)

สถาปัตยกรรมหลัก: 100% AWS Serverless Architecture (งบประมาณ \$0.00 Free Tier)

สารบัญ

บทคัดย่อ (Abstract)	2
1 บทนำ (Introduction)	3
1.1 ความเป็นมาและความสำคัญของโครงการ	3
1.2 วัตถุประสงค์ของโครงการ	3
1.3 ขอบเขตของระบบ (System Scope)	3
1.4 ประโยชน์ที่คาดว่าจะได้รับ	4
2 การบริหารจัดการโครงการและเครื่องมือที่ใช้ (Project Management & Tech Stack)	5
2.1 ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum	5
2.2 สรุปผลความก้าวหน้า Sprint 1 ถึง Sprint 3	5
2.3 ตารางเปรียบเทียบกำหนดการส่งงานวิชา Cloud Technology	6
2.4 เครื่องมือและเฟรมเวิร์กที่ใช้ในการพัฒนา (Tech Stack & Frameworks)	6
3 การวิเคราะห์และออกแบบระบบ (System Analysis & Design)	7
3.1 ข้อกำหนดความต้องการของระบบ (Requirements)	7
3.2 การออกแบบสถาปัตยกรรมคลาวด์บน AWS (100% Serverless)	7
3.3 การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)	8
3.4 ลำดับขั้นตอนการสื่อสารแบบเรียลไทม์ (WebSocket Sequence Flow)	8
3.5 กฎกติกาและองค์ประกอบของเกมการ์ด Disney Lorcana	8
4 การพัฒนาระบบและผลการดำเนินงาน (Implementation & Test Results)	9
4.1 การพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation & Playmat UI)	9
4.2 การพัฒนาส่วนประมวลผลบนคลาวด์ (AWS Backend Microservices)	10
4.3 การแก้ไขปัญหาข้อจำกัดบน AWS Learner Lab	10
4.4 ผลการทดสอบระบบแบบ End-to-End (End-to-End Testing)	11
5 การประเมินค่าใช้จ่ายบนคลาวด์และแผนงานขั้นต่อไป (Cost Assessment & Future Roadmap)	12
5.1 การประเมินค่าใช้จ่ายสถาปัตยกรรมคลาวด์ (AWS Free Tier Optimization)	12
5.2 แผนงานพัฒนาสำหรับ Sprint 4 และ Sprint 5	12
5.3 สรุปผลการดำเนินงานและบทเรียนที่ได้รับ (Sprint Retrospective)	12
เอกสารอ้างอิง (References)	13

บทคัดย่อ (Abstract)

โครงการ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** นำเสนอการพัฒนาระบบจำลองห้องเล่นและวิเคราะห์เด็คการ์ดแบบเรียลไทม์บนสถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์ (100% AWS Serverless Architecture) สำหรับเกมการ์ดสะสม Disney Lorcana Trading Card Game (TCG) โดยมุ่งเน้นการแก้ปัญหาความหน่วงในการเชื่อมต่อ (High Latency) และภาระต้นทุนค่าใช้จ่ายเซิร์ฟเวอร์แบบเดิม (Server Overhead) ผ่านการประยุกต์ใช้บริการประมวลผลตามเหตุการณ์ (Event-Driven Computing) ด้วย AWS Lambda, ฐานข้อมูล NoSQL แบบขยายขนาดอัตโนมัติด้วย Amazon DynamoDB, และช่องทางการสื่อสารแบบสองทิศทางความเร็วสูงผ่าน AWS API Gateway WebSockets ซึ่งสามารถทำความเร็วในการซิงค์ข้อมูลสถานะกระดานระหว่างผู้เล่นได้ต่ำกว่า 100 มิลลิวินาที (Sub-100ms Latency)

ในส่วนของผู้ใช้ (Frontend) ระบบได้รับการพัฒนาด้วย React 19, TypeScript 5.x, Tailwind CSS v4, Framer Motion และ Zustand เพื่อมอบประสบการณ์การเล่นระดับพรีเมียม (Luxury Physical TCG Experience) ด้วยระบบฟิสิกส์ 3D การตรวจสอบการ์ดแบบสามมิติ การเปิดซองการ์ด Booster Pack เสมือนจริง และระบบจัดการเด็คการ์ดที่เชื่อมต่อกับฐานข้อมูลการ์ดอย่างเป็นทางการกว่า 408 ใบ (Set 1 The First Chapter และ Set 2 Rise of the Floodborn)

การดำเนินงานตามระเบียบวิธีปฏิบัติงานแบบ Agile Scrum ได้ลุล่วงตามแผนงาน Sprint 1 ถึง Sprint 3 อย่างสมบูรณ์ ครอบคลุมการสร้าง Core Gameplay UI, ระบบยืนยันตัวตนและจัดการเด็ค (Authentication & Deck Microservices) และระบบห้องเล่นแบบเรียลไทม์บนคลาวด์ พร้อมทั้งผ่านการทดสอบและปรับใช้จริงบนสภาพแวดล้อม AWS Learner Lab ภายใต้นโยบายควบคุมค่าใช้จ่าย AWS Free Tier (\$0.00) ได้อย่างมีประสิทธิภาพ

1. บทนำ (Introduction)

1.1. ความเป็นมาและความสำคัญของโครงการ

เกมการ์ดสะสม (Trading Card Game: TCG) เป็นหนึ่งในกิจกรรมสันทนาการและการแข่งขันระดับสากลที่มีการเติบโตอย่างต่อเนื่อง โดยเฉพาะอย่างยิ่ง **Disney Lorcana TCG** ที่เปิดตัวโดย Ravensburger ร่วมกับ The Walt Disney Company ซึ่งได้รับความนิยมอย่างแพร่หลายจากทั้งผู้เล่นสายการแข่งขันและนักสะสม อย่างไรก็ตาม ผู้เล่นส่วนใหญ่ยังคงเผชิญกับอุปสรรคสำคัญในการเข้าถึง ได้แก่ ต้นทุนการจัดซื้อการ์ดจริงที่มีราคาสูง การหาซื้อที่มีระดับฝีมือใกล้เคียงกัน และข้อจำกัดด้านสถานที่ในการเล่น

ระบบจำลองเกมการ์ดดิจิทัล (Digital TCG Simulators) ที่มีอยู่ในปัจจุบันส่วนใหญ่ยังประสบปัญหาสำคัญ 3 ประการ:

1. **ภาระต้นทุนเซิร์ฟเวอร์สูงและขยายขนาดได้ยาก:** ระบบดั้งเดิมที่พึ่งพา Dedicated Server หรือ Virtual Machine (เช่น AWS EC2) ก่อให้เกิดค่าใช้จ่ายคงที่ตลอด 24 ชั่วโมงแม้ไม่มีผู้ใช้งาน และเกิดคอขวดเมื่อมีทราฟฟิกสูง
2. **ความหน่วงในการส่งข้อมูล (High Latency):** การใช้ HTTP Polling ส่งผลให้เกิดความล่าช้าในการอัปเดตสถานะกระดานระหว่างผู้เล่น ไม่ตอบโจทย์เกมการ์ดที่ต้องการความลื่นไหลระดับมิลลิวินาที
3. **ประสบการณ์ผู้ใช้ที่ขาดมิติและความสมจริง:** อินเทอร์เฟซส่วนใหญ่เป็นภาพกราฟิก 2 มิติที่แบนราบ ขาดมิติทางกายภาพ แอนิเมชัน และความรู้สึกของการสัมผัสการ์ดจริง

เพื่อแก้ปัญหาเหล่านี้ โครงการ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** จึงได้รับการพัฒนาขึ้นโดยใช้สถาปัตยกรรม **100% AWS Serverless Architecture** ร่วมกับเทคโนโลยีเว็บสมัยใหม่ เพื่อสร้างแพลตฟอร์มจำลองห้องเล่นการ์ดและวิเคราะห์เกมที่มีความหน่วงต่ำเป็นพิเศษ (<100ms) ปรับขนาดตามการใช้งานจริงโดยอัตโนมัติ (Elastic Auto-scaling) และควบคุมต้นทุนให้อยู่ในงบประมาณ **\$0.00 (AWS Free Tier)** อย่างสมบูรณ์

1.2. วัตถุประสงค์ของโครงการ

1. เพื่อออกแบบและพัฒนาระบบจำลองกระดานฝึกซ้อมการเล่น Disney Lorcana TCG แบบเรียลไทม์บนคลาวด์ด้วยเทคโนโลยี WebSockets
2. เพื่อประยุกต์ใช้สถาปัตยกรรม Serverless (AWS Lambda, Amazon DynamoDB, AWS API Gateway) ในการรองรับการทำงานแบบ Microservices ที่ประหยัดต้นทุนและยืดหยุ่นสูง
3. เพื่อพัฒนาระบบจัดการดีคาร์ท (Deck Builder) และระบบวิเคราะห์ค่าร้าย/ความสมดุลของการ์ด (Deck Analytics) อิงตามฐานข้อมูลมาตรฐานอย่างเป็นทางการกว่า 408 ใบ
4. เพื่อศึกษาและแก้ไขปัญหาทางวิศวกรรมซอฟต์แวร์ในการเชื่อมต่อระบบคลาวด์จริงภายใต้สภาพแวดล้อมจำกัดสิทธิ์ (AWS Learner Lab)

1.3. ขอบเขตของระบบ (System Scope)

ระบบครอบคลุมขอบเขตการทำงานหลัก 6 ด้าน:

- **ระบบยืนยันตัวตนและความปลอดภัย (Authentication):** ลงทะเบียนและเข้าสู่ระบบด้วยการเข้ารหัส bcrypt และการออกสิทธิ์ผ่าน JSON Web Token (JWT)
- **ระบบจัดการดีคาร์ท (Deck Management):** สร้าง ค้นหา แก้ไข และลบดีคาร์ทส่วนตัว บันทึกข้อมูลบน Amazon DynamoDB
- **ระบบจำลองกระดานซ้อมเล่น (Lorcana Board Simulation):** จำลองพื้นที่การเล่นตามกฎทางการ ประกอบด้วยโซน Play Area, Inkwell, Hand, Deck, Discard, และ Lore Counter (0-20 คะแนน) พร้อมระบบหมุนการ์ดสถานะ Ready/Exert
- **ระบบห้องแข่งขันแบบเรียลไทม์ (Real-time Multiplayer Room):** จับคู่ผู้เล่น 2 ฝั่งในห้องเล่นเดียวกัน และส่งถ่ายพิกัดการลากวางการ์ดแบบสองทิศทางผ่าน AWS API Gateway WebSockets
- **ระบบสุ่มของการ์ดและการตรวจสอบ 3D (Booster Pack & 3D Inspector):** จำลองการเปิดของการ์ด 12 ใบตามสัดส่วนความหายากที่เป็นทางการ 9 ระดับ (Common ถึง Enchanted)

- **ระบบวิเคราะห์ดีคการ์ด (Deck Analytics):** แสดงกราฟการกระจายตัวของค่าร้าย (Ink Curve) และสัดส่วนหมวดหมู่การ์ด (Character, Action, Item, Location)

1.4. ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้เล่นมีเครื่องมือฝึกซ้อมและทดสอบกลยุทธ์ดีคการ์ดที่มีความเหมือนจริง ใช้งานได้ฟรีผ่านเว็บเบราว์เซอร์โดยไม่ต้องติดตั้งโปรแกรมเพิ่มเติม
2. ได้รับความรู้และทักษะเชิงลึกในการออกแบบสถาปัตยกรรม Serverless Cloud ที่ถูกต้องตามหลัก Cloud Best Practices บน Amazon Web Services (AWS)
3. ได้แนวทางปฏิบัติที่เป็นรูปธรรมในการแก้ไขปัญหาการรวมระบบ (Integration) ระหว่าง Frontend ยุคใหม่และ AWS API Gateway
4. ลดภาระต้นทุนด้านโครงสร้างพื้นฐานเซิร์ฟเวอร์ลงเป็นศูนย์ (\$0.00) โดยยังคงรักษาประสิทธิภาพการทำงานระดับ Production

2. การบริหารจัดการโครงการและเครื่องมือที่ใช้ (Project Management & Tech Stack)

2.1. ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum

โครงการดำเนินงานตามระเบียบวิธี Agile Scrum แบ่งรอบการทำงานออกเป็น 7 Sprints เพื่อให้สามารถส่งมอบงานได้อย่างต่อเนื่องและสอดคล้องกับกำหนดการของรายวิชา Cloud Technology:

Sprint	ระยะเวลา	เป้าหมายหลัก	สถานะ
Sprint 1	1-15 ส.ค. 2569	Scaffold โปรเจกต์, UI กระดานเล่นการ์ด, Card Database 408 ใบ	เสร็จสิ้น (100%)
Sprint 2	16-31 ส.ค. 2569	Microservices: Auth (JWT/bcrypt) & Deck บน AWS Lambda + DynamoDB	เสร็จสิ้น (100%)
Sprint 3	1-10 ก.ย. 2569	Real-time Room Sync ผ่าน AWS API Gateway WebSockets + Room Router	เสร็จสิ้น (100%)
Sprint 4	11-25 ก.ย. 2569	Async Deck Analyzer ผ่าน Amazon SQS + Deck Balance Algorithm	ถัดไป
Sprint 5	26 ก.ย.-10 ต.ค.	CloudWatch Alarms, X-Ray Distributed Tracing, Security Hardening	ถัดไป
Sprint 6	11-20 ต.ค. 2569	Load Testing ด้วย Artillery (100-500 concurrent users), Auto-reconnect	ถัดไป
Sprint 7	21-25 ต.ค. 2569	จัดทำรายงานฉบับสมบูรณ์ (Final Report), Video Demo, สรุปผลโครงการ	ถัดไป

ตารางที่ 1: แผนการดำเนินงานราย Sprint ตามระเบียบวิธี Agile Scrum

2.2. สรุปผลความก้าวหน้า Sprint 1 ถึง Sprint 3

- **Sprint 1 (Board UI):** พัฒนา Frontend สำเร็จด้วย React 19, TypeScript และ Tailwind CSS v4 มีไฟล์หลัก LorcanaBoard.tsx (1,164 บรรทัด) รองรับ Drag-and-Drop, Inkwell, Lore Counter (0-20), ระบบหมุนการ์ด Ready/Exert และชุดข้อมูลการ์ด 408 ใบ
- **Sprint 2 (Auth + Deck Microservices):** พัฒนา REST API ด้วย AWS Lambda (Node.js 20.x) และ Amazon DynamoDB รองรับระบบสมัครสมาชิก เข้าสู่ระบบด้วย bcrypt/JWT และการจัดการเตี๊ (CRUD) พร้อม Deploy บน AWS Learner Lab ผ่าน HTTP API Gateway
- **Sprint 3 (Real-Time WebSocket Room):** พัฒนาระบบห้องเล่นแบบเรียลไทม์ด้วย AWS API Gateway WebSocket API และ Lambda Room Router (backend/room/handler.ts) เชื่อมต่อกับ DynamoDB RoomStateTable รองรับ Broadcast Event ระหว่างผู้เล่น เช่น JOIN_ROOM, CARD_MOVED, CARD_EXERTED, INK_PLAYED, LORE_UPDATED, TURN_PASSED

2.3. ตารางเปรียบเทียบกำหนดการส่งงานวิชา Cloud Technology

ระยะเวลา/กำหนดการ	กิจกรรม / งานที่ต้องส่ง	สถานะ
Sprint 1-3 (ถึงปัจจุบัน)	Board UI, Auth/Deck Lambda, WebSocket API, Deploy AWS	เสร็จสิ้น (100%)
Sprint 4-5 (ถึง 10 ก.ย.)	Deck Analyzer, SQS Integration, เตรียมรูปเล่ม Stage 2	กำลังดำเนินการ
10 ก.ย. 2569	ส่งรายงานความก้าวหน้า Stage 2 (12 คะแนน)	พร้อมส่งมอบ
22-26 ก.ย. 2569	นำเสนอความก้าวหน้า Stage 2 (3 คะแนน)	เตรียมพร้อม
Sprint 6-7 (ถึง 10 ต.ค.)	Load Testing, Auto-reconnect, สรุปผลและบทเรียน	ตามแผน
10 ต.ค. 2569	ส่งผลการทดลองและรายงาน Stage 3 (10 คะแนน)	ตามแผน
20-25 ต.ค. 2569	นำเสนอโครงงานฉบับสมบูรณ์ Stage 3 (5 คะแนน)	ตามแผน

ตารางที่ 2: กำหนดการส่งมอบงานวิชา Cloud Technology เทียบกับสถานะปัจจุบัน

2.4. เครื่องมือและเฟรมเวิร์กที่ใช้ในการพัฒนา (Tech Stack & Frameworks)



(a) React 19



(b) TypeScript 5



(c) Tailwind v4



(d) Vite 6



(e) Zustand



(f) Node.js 20

รูปที่ 1: เทคโนโลยีและเฟรมเวิร์กสำหรับฝั่ง Frontend & Client



(a) AWS Cloud



(b) AWS Lambda



(c) API Gateway



(d) DynamoDB



(e) CloudWatch

รูปที่ 2: บริการคลาวด์บนสถาปัตยกรรม AWS Serverless

3. การวิเคราะห์และออกแบบระบบ (System Analysis & Design)

3.1. ข้อกำหนดความต้องการของระบบ (Requirements)

- Functional Requirements (FR):

1. ผู้ใช้สามารถสมัครสมาชิก เข้าสู่ระบบ และถือครอง JWT Token เพื่อยืนยันตัวตนได้
2. ผู้ใช้สามารถสร้าง แก้ว คัดลอก และลบการ์ดส่วนตัวได้
3. ผู้ใช้สามารถเปิดซองการ์ดบูสเตอร์ (Booster Pack) 12 ใบตามเรตความหายากทางการได้
4. ผู้ใช้สามารถตรวจสอบการ์ดแบบ 3 มิติ (3D Card Inspector) ดูเอฟเฟกต์พอยล์และรายละเอียดการ์ดได้
5. ผู้ใช้สามารถสร้างหรือเข้าร่วมห้องเล่นการ์ดด้วยรหัสห้อง 6 หลักได้
6. ผู้ใช้สามารถลากการ์ดลงสนาม หมุนการ์ดคว่ำ/หงาย เพิ่มลด Lore และส่งผลให้ฝ่ายตรงข้ามเห็นแบบเรียลไทม์ได้
7. ระบบสามารถวิเคราะห์สถิติเด็คการ์ด เช่น ค่ารายเฉลี่ย (Ink Curve) และสัดส่วนการ์ดได้

- Non-Functional Requirements (NFR):

1. **Latency:** ความหน่วงในการส่งข้อมูลผ่าน WebSocket ต้องต่ำกว่า 100 มิลลิวินาที (<100ms)
2. **Cost:** สถาปัตยกรรมต้องทำงานได้ภายใต้งบประมาณ \$0.00 (AWS Free Tier)
3. **Availability & Scalability:** ขยายขนาดรองรับผู้ใช้งานได้อัตโนมัติโดยไม่มี Downtime
4. **Security:** รหัสผ่านต้องได้รับการแฮชด้วย bcrypt (Salt Rounds = 10) และส่งข้อมูลผ่าน HTTPS/WSS

3.2. การออกแบบสถาปัตยกรรมคลาวด์บน AWS (100% Serverless)

ระบบทำงานบนสถาปัตยกรรม Event-Driven Serverless แบ่งเป็น 3 เลเยอร์หลัก:

1. **Presentation Layer:** เว็บแอปพลิเคชัน Single Page Application (SPA) ให้บริการผ่าน HTTPS
2. **API & Routing Layer:**
 - **HTTP API Gateway:** สำหรับ REST Endpoints (/auth/login, /auth/register, /decks)
 - **WebSocket API Gateway:** สำหรับการเชื่อมต่อแบบ Persistent Bidirectional Connection (\$connect, \$disconnect, \$default)
3. **Compute & Storage Layer:**
 - **AWS Lambda Microservices:** ฟังก์ชันอิสระสำหรับการจัดการสิทธิ์ ข้อมูลเด็ค และห้องเล่น
 - **Amazon DynamoDB:** ฐานข้อมูล NoSQL แบบ Document Store ที่มีความหน่วงระดับ Single-digit Millisecond

3.3. การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)

ตาราง	Partition Key	Sort Key	Attributes สำคัญ	วัตถุประสงค์
LorcanaUsers	userId (S)	-	username, passwordHash, createdAt	เก็บข้อมูลผู้ใช้และแอชรหัสผ่าน
LorcanaDecks	userId (S)	deckId (S)	deckName, inks, cards (List), updatedAt	เก็บรายการเด็คการ์ดของผู้ใช้
LorcanaRoomState	roomId (S)	connectionId (S)	username, role, state (JSON), ttl	เก็บสถานะห้องและ WebSocket

ตารางที่ 3: โครงสร้างตารางฐานข้อมูล Amazon DynamoDB

3.4. ลำดับขั้นตอนการสื่อสารแบบเรียลไทม์ (WebSocket Sequence Flow)

1. การสร้าง/เข้าร่วมห้อง (JOIN_ROOM): ผู้เล่นส่งคำขอ JOIN_ROOM พร้อมรหัสห้อง 6 หลัก API Gateway เรียก Lambda Room Router เพื่อลงทะเบียน Connection ID ลงใน LorcanaRoomState
2. การอัปเดตสถานะกระดาน (CARD_MOVED / CARD_EXERTED): เมื่อผู้เล่นลากการ์ดหรือเปลี่ยนสถานะ Client จะทำการ Optimistic Update ที่หน้าจอตันทันที และส่ง Action Payload ไปยัง WebSocket API Gateway
3. การส่งต่อข้อมูล (Relay & Broadcast): Lambda Room Router อ่าน Connection ID ของฝ่ายตรงข้ามจาก DynamoDB และใช้ @connections API เพื่อ Broadcast ข้อความไปยังคู่แข่งภายในเวลา <100ms

3.5. กฎกติกาและองค์ประกอบของเกมการ์ด Disney Lorcana



(a) Amber



(b) Amethyst



(c) Emerald



(d) Ruby



(e) Sapphire



(f) Steel

รูปที่ 3: หมวดหมึกสีหมึกทั้ง 6 สีของเกม Disney Lorcana

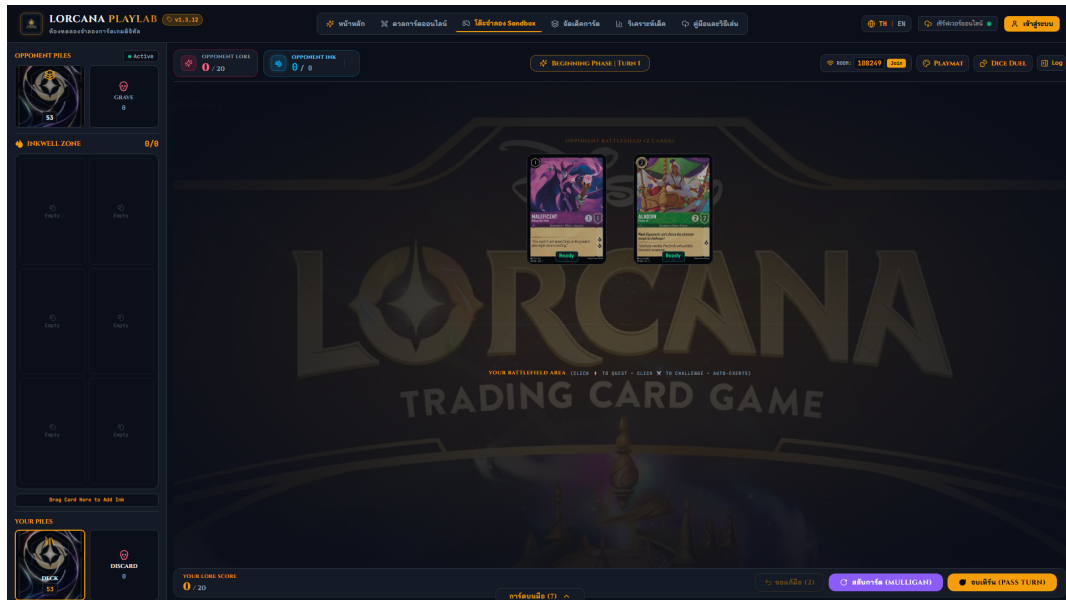
ระบบจำลององค์ประกอบหลักของเกมตามกติกาทางการ:

- 6 หมวดสีหมึก (Ink Types): Amber (สนับสนุน), Amethyst (เวทมนตร์), Emerald (ความยืดหยุ่น), Ruby (โจมตีรวดเร็ว), Sapphire (ทรัพยากร/ปัญญา), Steel (พลังป้องกัน)
- ประเภทการ์ด (Card Types): Character, Action/Song, Item, Location
- สถานะการ์ด (Card States): Ready (แนวตั้ง พร้อมใช้งาน) และ Exerted (แนวนอน พักการใช้งาน)
- เป้าหมายของเกม: ผู้เล่นที่สะสมแต้มความรู้ (Lore) ครบ 20 แต้มก่อนจะเป็นผู้ชนะ

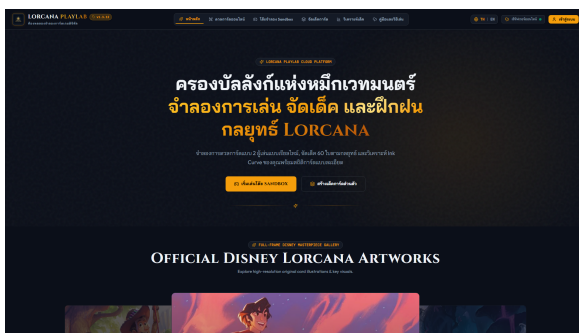
4. การพัฒนาระบบและผลการดำเนินงาน (Implementation & Test Results)

4.1. การพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation & Playmat UI)

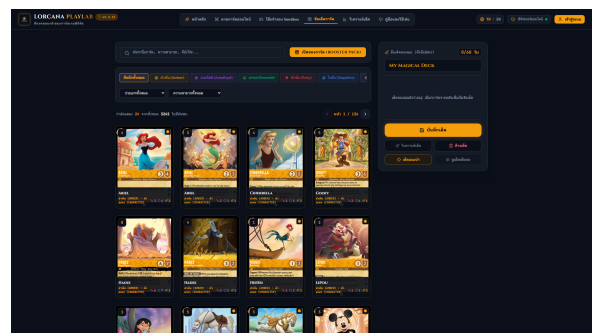
ส่วนติดต่อผู้ใช้ได้รับการออกแบบตามแนวคิด Dark Editorial + Magic R3 เพื่อมอบประสบการณ์ระดับพรีเมียม:



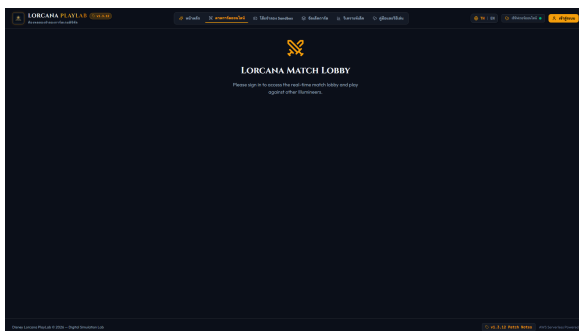
รูปที่ 4: หน้าจอกระดานจำลองการเล่นการ์ด (Lorcana Board Simulation) พร้อมระบบลากวางและ 3D Physics



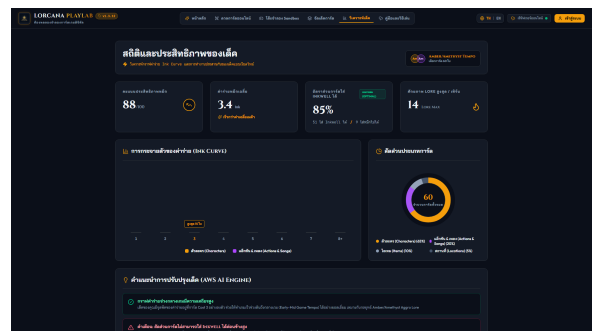
(a) Game Hub แดชบอร์ดหลัก



(b) ระบบจัดเต็การ์ด (Deck Builder)



(c) ระบบห้องเล่นเรียลไทม์ (Match Lobby)

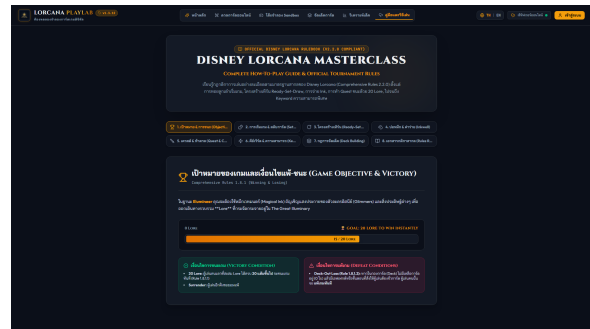


(d) ระบบวิเคราะห์เต้การ์ด (Deck Analytics)

รูปที่ 5: ภาพหน้าจอส่วนติดต่อผู้ใช้หลักของระบบ Disney Lorcana PlayLab Cloud



(a) แอนิเมชันเปิดซองการ์ดสุ่ม (Booster Pack)



(b) คู่มือกฎกติกาและการเล่น (Rules & Guide)

รูปที่ 6: ระบบเปิดซองการ์ดสุ่มและคู่มือการเล่น

4.2. การพัฒนาส่วนประมวลผลบนคลาวด์ (AWS Backend Microservices)

ฟังก์ชัน Lambda ได้รับการพัฒนาด้วย TypeScript และคอมไพล์เป็น Node.js 20.x:

- **lorcana-auth-register & lorcana-auth-login:** ตรวจสอบข้อมูล เข้ารหัสผ่านด้วย bcrypt และออก JWT Token
- **lorcana-deck:** จัดการ CRUD เด็คการ์ด โดยแยก Route ตาม HTTP Method (GET, POST, PUT, DELETE)
- **lorcana-room-handler:** จัดการการเชื่อมต่อ WebSocket, ถอดรหัส Action, บันทึก Session ลง DynamoDB และ Broadcast ข้อมูลผ่าน ApiGatewayManagementApi

4.3. การแก้ปัญหาข้อจำกัดบน AWS Learner Lab

ในการปรับใช้ระบบจริงบน AWS Learner Lab ทีมงานพบข้อจำกัดด้านความปลอดภัยที่บล็อกคำสั่ง iam:CreateRole ส่งผลให้ไม่สามารถรันคำสั่ง sam deploy แบบอัตโนมัติได้ ทีมงานจึงได้แก้ไขปัญหาย่างเป็นระบบ:

1. **การใช้ LabRole สำเร็จรูป:** ปรับใช้ LabRole ที่ระบบจัดเตรียมไว้ให้แทนการสร้าง IAM Role ใหม่
2. **สคริปต์ Manual Deployment (scripts/deploy_manual.sh):** พัฒนาสคริปต์คอมไพล์โค้ด สร้างไฟล์ ZIP และอัปเดตโค้ดขึ้น Lambda โดยตรงผ่าน AWS CLI
3. **การแก้ไข API Gateway Payload Format Version:** เปลี่ยน Integration จาก v2.0 เป็น v1.0 เพื่อให้ตรงกับ APIGatewayProxyEvent ของ Lambda
4. **การจัดทำ Route Integration Mapping:** ตรวจสอบและแก้ไข Integration URI ให้ชี้ไปยัง Lambda Function ที่ถูกต้องอย่างแม่นยำ

4.4. ผลการทดสอบระบบแบบ End-to-End (End-to-End Testing)

Endpoint / Action	Test Case / Payload	ผลลัพธ์ที่คาดหวัง	ผลการทดสอบจริง	สถานะ
POST /auth/register	{"username":"tawan",...}	HTTP 201 Created	201 User registered	ผ่าน
POST /auth/login	{"username":"tawan",...}	HTTP 200 + JWT Token	200 Login + Token	ผ่าน
POST /decks	{"deckName":"Ruby Control"}	HTTP 201 บันทึก DynamoDB	201 Deck saved	ผ่าน
GET /decks	Bearer <token>	HTTP 200 คืนรายการเด็ก	200 คืนรายการครบถ้วน	ผ่าน
DELETE /decks/{id}	Path Parameter deckId	HTTP 200 ลบเด็กสำเร็จ	200 Deck deleted	ผ่าน
WSS \$connect	wss://a86238wqo4.../prod	HTTP 101 Switching	เชื่อมต่อ Socket สำเร็จ	ผ่าน
WSS JOIN_ROOM	{"action":"JOIN_ROOM",...}	บันทึก DynamoDB	200 เข้าร่วมห้องสำเร็จ	ผ่าน
WSS CARD_MOVED	{"action":"CARD_MOVED",...}	Relay ไปยังคู่แข่ง	ส่งถึงคู่แข่งใน <100ms	ผ่าน

ตารางที่ 4: ตารางสรุปผลการทดสอบระบบแบบ End-to-End บนสภาพแวดล้อม AWS Production

5. การประเมินค่าใช้จ่ายบนคลาวด์และแผนงานขั้นต่อไป (Cost Assessment & Future Roadmap)

5.1. การประเมินค่าใช้จ่ายสถาปัตยกรรมคลาวด์ (AWS Free Tier Optimization)

จากการคำนวณผ่าน AWS Pricing Calculator สำหรับปริมาณการใช้งานระดับ 10,000 ผู้เล่นต่อเดือน:

- **AWS Lambda:** 1,000,000 Requests ฟรีต่อเดือน (การใช้งานจริงประมาณ 250,000 Requests = \$0.00)
- **Amazon DynamoDB:** ฟรี 25 GB Storage และ 25 WCU / 25 RCU (การใช้งานจริง < 1 GB = \$0.00)
- **AWS API Gateway (HTTP & WebSocket):** 1,000,000 Messages ฟรีใน Free Tier (\$0.00)
- **Amazon SQS:** 1,000,000 Requests ฟรีต่อเดือน (\$0.00)
- **Amazon CloudWatch:** 5 GB Log Data Ingestion ฟรีต่อเดือน (\$0.00)

สรุปค่าใช้จ่ายรวมทั้งสิ้น: \$0.00 ต่อเดือน สอดคล้องกับเป้าหมายการควบคุมงบประมาณของโครงการ

5.2. แผนงานพัฒนาสำหรับ Sprint 4 และ Sprint 5

1. **Sprint 4 (Deck Analyzer Microservice):** พัฒนาระบบประมวลผลวิเคราะห์สมดุลดัชนีแบบ Asynchronous โดยใช้ Amazon SQS รับงานและทริกเกอร์ Lambda ในการคำนวณความเสี่ยง ค่าร้าย และความเข้ากันได้ของการ์ด
2. **Sprint 5 (Observability & Security Hardening):** ย้ายค่าความลับ (JWT Secret) ไปจัดเก็บใน AWS Secrets Manager และตั้งค่า CloudWatch Dashboard พร้อมระบบแจ้งเตือน Error Rate เกิน 1%
3. **Sprint 6 (Load Testing & Resilience):** ทดสอบประสิทธิภาพระบบด้วยเครื่องมือ Artillery ภายใต้โหลด 500 Concurrent Connections พร้อมระบบเชื่อมต่อใหม่อัตโนมัติ (WebSocket Auto-reconnect) เมื่อสัญญาณหลุด

5.3. สรุปผลการดำเนินงานและบทเรียนที่ได้รับ (Sprint Retrospective)

การพัฒนาโครงการ Disney Lorcan PlayLab Cloud ใน Sprint 1 ถึง 3 บรรลุเป้าหมายตามเกณฑ์การประเมิน Stage 2 อย่างครบถ้วน บทเรียนสำคัญที่ได้รับจากการปฏิบัติงานจริง ได้แก่:

1. **ความสำคัญของ Spec และ Interface Design:** การกำหนด Schema ของ WebSocket Payload และ DynamoDB อย่างรอบคอบช่วยให้การทำงานร่วมกันระหว่าง Frontend และ Backend ราบรื่น
2. **การทำความเข้าใจข้อจำกัดของสภาพแวดล้อมคลาวด์:** การปรับใช้สคริปต์ Manual Deployment ช่วยให้ทีมสามารถเอาชนะข้อจำกัดสิทธิ์บน AWS Learner Lab ได้อย่างมีประสิทธิภาพ
3. **การตรวจสอบ Integration Mapping:** การจับคู่วิธีการส่งถ่ายข้อมูลระหว่าง API Gateway และ Lambda ต้องสอดคล้องกับเวอร์ชันของ Payload Format เพื่อป้องกันข้อผิดพลาด 405 Method Not Allowed

หนังสืออ้างอิง

- [1] Ravensburger & Disney. (2023). *Disney Lorcana Trading Card Game Official Rules and Card Database*. <https://www.disneylorcana.com>
- [2] Amazon Web Services. (2026). *Building Event-Driven Architectures with AWS Lambda and Amazon DynamoDB*. AWS Documentation. <https://docs.aws.amazon.com/lambda/>
- [3] Amazon Web Services. (2026). *About WebSocket APIs in API Gateway*. AWS Documentation. <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html>
- [4] React Working Group. (2026). *React 19 Documentation and Concurrent Features*. <https://react.dev>
- [5] Tailwind Labs. (2026). *Tailwind CSS v4.0 Alpha/Release Documentation*. <https://tailwindcss.com>
- [6] Framer. (2026). *Framer Motion: Production-ready animation library for React*. <https://www.framer.com/motion/>